

LAPORAN PRAKTIKUM JARINGAN KOMPUTER - MODUL 6

Transmission Control Protocol (TCP) Analysis

Identitas Mahasiswa

Nama: Jevico Royvaldo

NIM: 103072400151

Kelas: IF-04-01

A. Tujuan Praktikum

1. Mampu menginvestigasi dan menganalisis perilaku serta cara kerja protokol Transmission Control Protocol (TCP) melalui aplikasi Wireshark.
2. Memahami parameter TCP seperti nomor urutan (Sequence Number), Acknowledgement, serta mekanisme pengendalian kemacetan jaringan (Congestion Control).

B. Langkah Praktikum

1. Mengunduh file teks sampel berukuran besar (Alice in Wonderland) pada tautan <http://gaia.cs.umass.edu/wireshark-labs/alice.txt>.
2. Membuka portal pengunggahan berkas pada halaman <http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html>.
3. Mengaktifkan fitur perekaman paket (Capture) pada Wireshark dengan memilih antarmuka jaringan yang aktif (Wi-Fi).
4. Mengeksekusi tombol "Upload file alice.txt" pada browser untuk mengirimkan data ke server gaia.cs.umass.edu menggunakan metode HTTP POST.
5. Menghentikan proses perekaman paket sesaat setelah pesan konfirmasi keberhasilan muncul pada layar browser.
6. Menerapkan filter tampilan tcp pada Wireshark untuk mengisolasi paket data terkait.

C. Hasil dan Analisis Data

1. Informasi Dasar Koneksi TCP

No.	Time	Source	Destination	Protocol	Length	Info
754	29.514936700	2404:c0:b601:5d58:e...	2603:1046:1406:1:1:4	TCP	74	55689 → 443 [ACK] Seq=30 Ack=26 Win=1023 Len=0
755	29.530417400	192.168.1.48	57.144.193.32	TLSv1.2	124	Application Data
756	29.561086100	57.144.193.32	192.168.1.48	TCP	54	443 → 58854 [ACK] Seq=403 Ack=408 Win=365 Len=0
757	29.736516900	57.144.193.32	192.168.1.48	TLSv1.2	126	Application Data
758	29.777297200	192.168.1.48	57.144.193.32	TCP	54	58854 → 443 [ACK] Seq=408 Ack=475 Win=253 Len=0
761	30.905056000	2603:1063:5::fe	2404:c0:b601:5d58:e...	TLSv1.2	194	Application Data
762	30.906583800	2404:c0:b601:5d58:e...	2603:1063:5::fe	TLSv1.2	109	Application Data
763	30.919976600	2404:c0:b601:5d58:e...	2603:1063:5::fe	TLSv1.2	527	Application Data
764	30.969777500	2603:1063:5::fe	2404:c0:b601:5d58:e...	TCP	74	443 → 55329 [ACK] Seq=121 Ack=36 Win=651 Len=0
765	30.983511400	2603:1063:5::fe	2404:c0:b601:5d58:e...	TCP	74	443 → 55329 [ACK] Seq=121 Ack=489 Win=650 Len=0
766	30.983511400	2603:1063:5::fe	2404:c0:b601:5d58:e...	TLSv1.2	116	Application Data
767	30.985549900	2404:c0:b601:5d58:e...	2603:1063:5::fe	TLSv1.2	520	Application Data
771	31.091449600	2603:1063:5::fe	2404:c0:b601:5d58:e...	TCP	74	443 → 55329 [ACK] Seq=163 Ack=935 Win=651 Len=0
788	32.572289400	192.168.1.48	103.28.54.102	TLSv1.2	108	Application Data
789	32.632218500	103.28.54.102	192.168.1.48	TCP	54	27024 → 61318 [ACK] Seq=1 Ack=109 Win=1351 Len=0
793	33.768471600	192.168.1.48	128.119.245.12	TCP	687	64344 → 80 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=633 [TCP PDU reassembled in 946]
794	33.768693600	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=634 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
795	33.768693600	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=2086 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
796	33.768693600	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=3538 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
797	33.768693600	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=4990 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
798	33.768693600	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=6442 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
799	33.768693600	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=7894 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
800	33.768693600	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=9346 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
801	33.768693600	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=10798 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
802	33.768693600	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=12250 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
803	34.041632100	128.119.245.12	192.168.1.48	TCP	54	80 → 64344 [ACK] Seq=1 Ack=634 Win=63616 Len=0
804	34.041632100	128.119.245.12	192.168.1.48	TCP	54	80 → 64344 [ACK] Seq=1 Ack=2086 Win=67072 Len=0
805	34.041632100	128.119.245.12	192.168.1.48	TCP	54	80 → 64344 [ACK] Seq=1 Ack=3538 Win=70016 Len=0
806	34.041692700	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=13702 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]
807	34.041692700	192.168.1.48	128.119.245.12	TCP	1506	64344 → 80 [ACK] Seq=15154 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 946]

Berdasarkan investigasi paket yang berhasil ditangkap pada sesi praktikum mandiri, berikut adalah parameter identifikasi koneksi yang terbentuk:

- **Alamat IP Sumber (Komputer Klien):** 192.168.1.48
- **Nomor Port Sumber (TCP Source Port):** 64344
- **Alamat IP Tujuan (Server gaia.cs.umass.edu):** 128.119.245.12
- **Nomor Port Tujuan (TCP Destination Port):** 80 (Layanan Web HTTP)

2. Analisis Struktur Paket dan Urutan Data

a. Berapa nomor urut segmen TCP yang berisi perintah HTTP POST?

Berdasarkan rekaman pada baris Paket No. 793, segmen awal yang membawa instruksi aplikasi berupa metode HTTP POST memiliki nilai nomor urut (Sequence Number) relatif yaitu 1.

b. Sebutkan nomor urut (Sequence Number) dan waktu pengiriman dari 6 segmen TCP pertama yang dikirimkan oleh klien!

Merujuk pada runtutan paket data yang keluar dari komputer klien mulai dari Paket No. 793, berikut adalah rincian waktunya:

- **Segmen 1 (HTTP POST):** Sequence Number = 1 | Waktu Kirim = 33.768471600 detik
- **Segmen 2:** Sequence Number = 634 | Waktu Kirim = 33.768693600 detik

- **Segmen 3:** Sequence Number = 2086 | Waktu Kirim = 33.768693600 detik
- **Segmen 4:** Sequence Number = 3538 | Waktu Kirim = 33.768693600 detik
- **Segmen 5:** Sequence Number = 4990 | Waktu Kirim = 33.768693600 detik
- **Segmen 6:** Sequence Number = 6442 | Waktu Kirim = 33.768693600 detik

Analisis: Dapat diamati bahwa Segmen 2 hingga Segmen 6 dikirimkan secara simultan (bersama-sama) dalam fraksi milidetik yang sama setelah instruksi HTTP POST dieksekusi.

c. Berapa panjang data (Payload Length) dari masing-masing segmen tersebut?

- **Segmen 1:** Memiliki panjang payload bersih sebesar **633 Byte** (membawa header HTTP POST).
- **Segmen 2 sampai 6:** Memiliki panjang payload maksimal yang seragam sebesar **1452 Byte**. Ukuran 1452 Byte ini disesuaikan dengan nilai Maximum Segment Size (MSS) yang diizinkan oleh kartu jaringan lokal komputer klien guna menghindari fragmentasi.

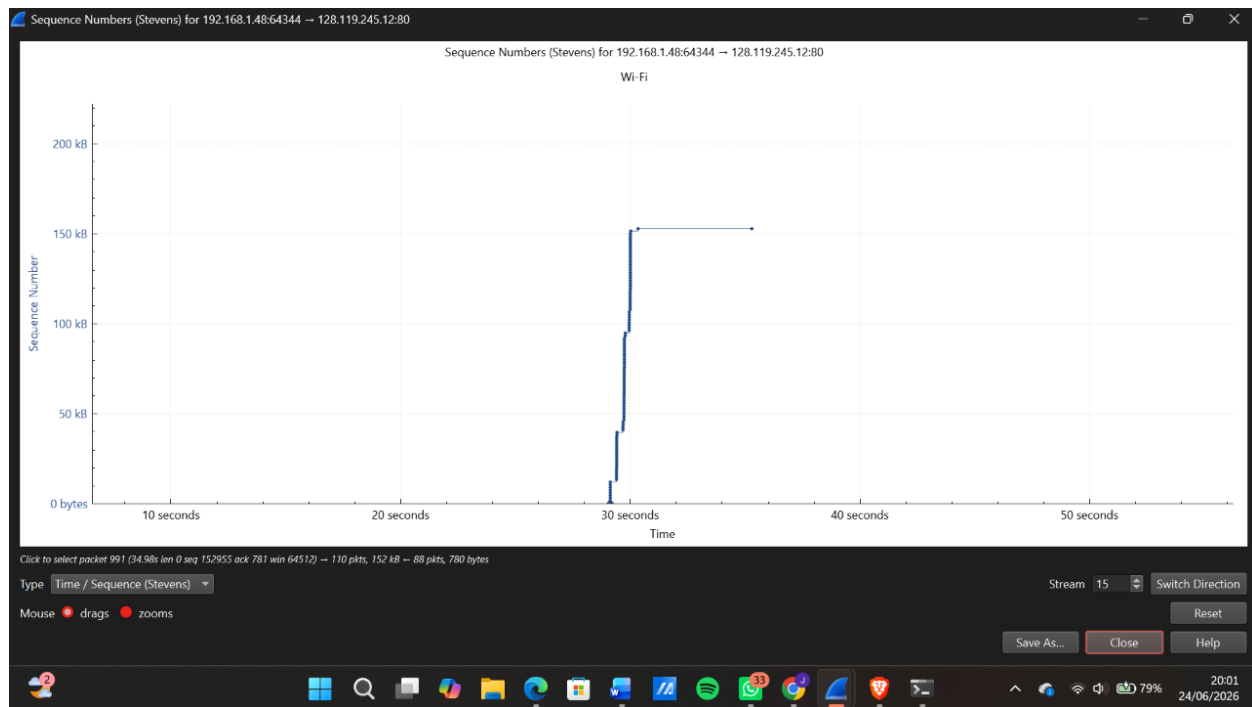
d. Berapa kapasitas buffer penerima (Window Size) minimum yang ditawarkan oleh server sepanjang koneksi? Apakah terdapat indikasi terjadinya hambatan akibat keterbatasan buffer?

Pada paket data awal (Paket 793), server menawarkan kapasitas jendela buffer penerima (Window Size) sebesar **65280 Byte**. Selama aktivitas transfer data berlangsung, nilai penawaran jendela buffer dari sisi server terpantau selalu berada pada kapasitas yang lapang dan aman. Oleh karena itu, tidak ditemukan adanya kendala kemacetan transmisi akibat keterbatasan ruang buffer penerima (Zero Window / Window Full).

e. Apakah terdapat indikasi adanya segmen yang mengalami transmisi ulang (Retransmission) akibat hilangnya paket?

Setelah dilakukan inspeksi menyeluruh pada baris log penangkapan paket, **tidak ditemukan adanya bendera ataupun indikasi TCP Retransmission**. Kondisi ini membuktikan bahwa jalur komunikasi antara perangkat komputer klien dan server tujuan berada dalam kondisi yang sangat stabil tanpa interferensi atau kehilangan paket (packet loss).

3. Analisis Pengendalian Kemacetan (TCP Congestion Control)



a. Lakukan analisis terhadap fase Slow Start dan Congestion Avoidance berdasarkan grafik Time/Sequence (Stevens) yang diperoleh!

- **Analisis Fase Slow Start:** Melalui grafik orisinal yang dihasilkan pada gambar, fase **Slow Start** terdeteksi berjalan sangat agresif sesaat setelah detik ke-30. Karakteristik ini ditandai dengan bentuk kurva nomor urutan (Sequence Number) yang langsung melonjak naik secara eksponensial (membentuk garis vertikal tajam ke atas) dari nilai 0 bytes hingga menembus batas penyerahan file mendekati nilai **150 kB**.
- **Analisis Fase Congestion Avoidance:** Pada perilaku ideal, kurva grafik seharusnya berubah melandai secara linear ketika algoritma memasuki fase Congestion Avoidance. Namun, pada hasil pengujian mandiri ini, fase linear tersebut tidak terlihat. Hal ini dikarenakan ukuran file alice.txt yang dikirimkan tergolong kecil (~150 KB), sehingga seluruh segmen data file tersebut telah sukses dikirimkan secara tuntas dalam hitungan milidetik sebelum jendela kongesti (Congestion Window) sempat menyentuh ambang batas perpindahan algoritma (sssthresh). Kurva kemudian langsung bergerak horizontal (mendatar) menandakan seluruh rangkaian transmisi data telah selesai seutuhnya.

D. Kesimpulan

Melalui pelaksanaan praktikum analisis protokol TCP menggunakan aplikasi Wireshark secara mandiri, didapatkan kesimpulan bahwa proses pengiriman data via metode HTTP POST telah berjalan dengan andal memanfaatkan port dinamis klien **64344** menuju port **80** pada server tujuan. Protokol TCP terbukti mampu menangani pembagian segmentasi data berukuran maksimum **1452 Byte** secara efisien serta melacak penerimaan data melalui skema nomor urut

(Sequence Number) yang konsisten. Hasil visualisasi grafik Stevens menunjukkan bahwa seluruh berkas data (~150 KB) berhasil ditransmisikan secara instan pada fase Slow Start tanpa adanya kendala paket hilang (retransmission) maupun hambatan memori buffer pada sisi penerima.